SEMESTER PROJECT-An Enterprise Relational Database for Conventional and Shariah-Compliant Retail Banking Operations



Submitted to: Maria Zulfiqar
Subject: Database Systems
Date: 30 March 2026.

Submitted by:
Muhammad Azeem

DEPARTMENT OF SOFTWARE ENGINEERING
UNIVERSITY OF LAHORE

Project Proposal: Online Banking Management System

Project Description:

The Simple Bank Management System is a basic database system used to store and manage customer and account information. It maintains records of customers, their accounts, and transactions such as deposits and withdrawals.
This system helps reduce manual record-keeping and allows quick access to data using SQL queries. It is designed for beginner-level understanding of DDL and DML commands

Functionalities

- Create table for customers
- Create table for accounts
- Create table for transactions
- Insert new customer
- Insert new account
- Insert transaction record
- Update customer information
- Update account balance
- Delete customer record
- Delete account record
- Search customer by ID
- Search customer by name
- Search account by account number
- Display all customers
- Display all accounts
- Deposit money

- Withdraw money
- View account balance
- View all transactions
- View transactions of a specific account

Target Users

- **Bank Staff:** Manage customer and account data easily.
- **Admin:** Monitor and control the system.
- **Customers (Indirectly):** Benefit from accurate and fast service.

Introduction:

The Online Banking Management System is designed to manage banking operations in Pakistan by providing digital services to customers. It allows users to create bank accounts, transfer funds, pay utility bills, and apply for loans online. The system supports local banking needs such as interbank transfers, branch-based services, and ATM usage. It helps banks improve efficiency, reduce manual work, and provide secure financial services across Pakistan.

Problem Statement

In Pakistan, many banking processes still involve manual work, long queues, and limited access to services, especially in smaller cities. Customers face delays in transactions, loan processing, and account management.
This system aims to:

- Digitize banking services
- Reduce dependency on physical branches
- Improve transaction speed and accuracy
- Provide easy access to financial services across Pakistan

Proposed Entities

1. **Customer** – Stores CNIC, name, phone number, and address
2. **Account** – Stores account number, balance, and account type
3. **Transaction** – Records deposits, withdrawals, and transfers

4. **Loan** – Stores loan applications and repayment details
5. **Branch** – Represents bank branches in different cities (Lahore, Karachi, etc.)
6. **Employee** – Stores bank staff information
7. **Card** – Manages ATM/debit card details
8. **ATM** – Stores ATM locations across Pakistan
9. **Payment** – Handles utility bills (electricity, gas, internet)
10. **Beneficiary** – Stores interbank transfer recipients
11. **Login** – Manages authentication (username, password)
12. **Notification** – Sends SMS/email alerts for transactions
13. **Account Type** – Savings, Current, Islamic account types

Core System Functionalities

- Customer registration using CNIC
- Secure login system
- Account creation and management
- Deposit and withdrawal
- Interbank fund transfer (IBAN-based)
- Utility bill payments (electricity, gas, internet)
- Loan application and tracking
- ATM and debit card management
- Add/manage beneficiaries
- View transaction history
- SMS/email notifications
- Admin control over branches and employees

User Roles and Access Levels

1. Admin

- Manage branches across Pakistan
- Manage employees
- Monitor system activity

2. Employee

- Open customer accounts
- Approve/reject loans
- Assist customers in transactions

3. Customer

- Open and manage account
- Transfer money
- Pay bills
- Apply for loans
- Manage card details

Use Case Scenarios

Use Case 1: Interbank Fund Transfer

1. Customer logs into system
2. Adds beneficiary using IBAN
3. Enters transfer amount
4. System checks balance
5. Transaction is processed
6. SMS notification is sent

Use Case 2: Utility Bill Payment

1. Customer logs into account
2. Selects bill type (electricity/gas/internet)

3. Enters bill reference number
4. System fetches bill details
5. Customer confirms payment
6. Payment is completed and notification is sent

Assumptions and Limitations

- Internet access is required
- CNIC is mandatory for account creation
- System does not integrate with all banks (limited demo scope)
- Security features are basic for academic purposes
- Only supports Pakistani banking system

ER Diagram Explanation

Introduction

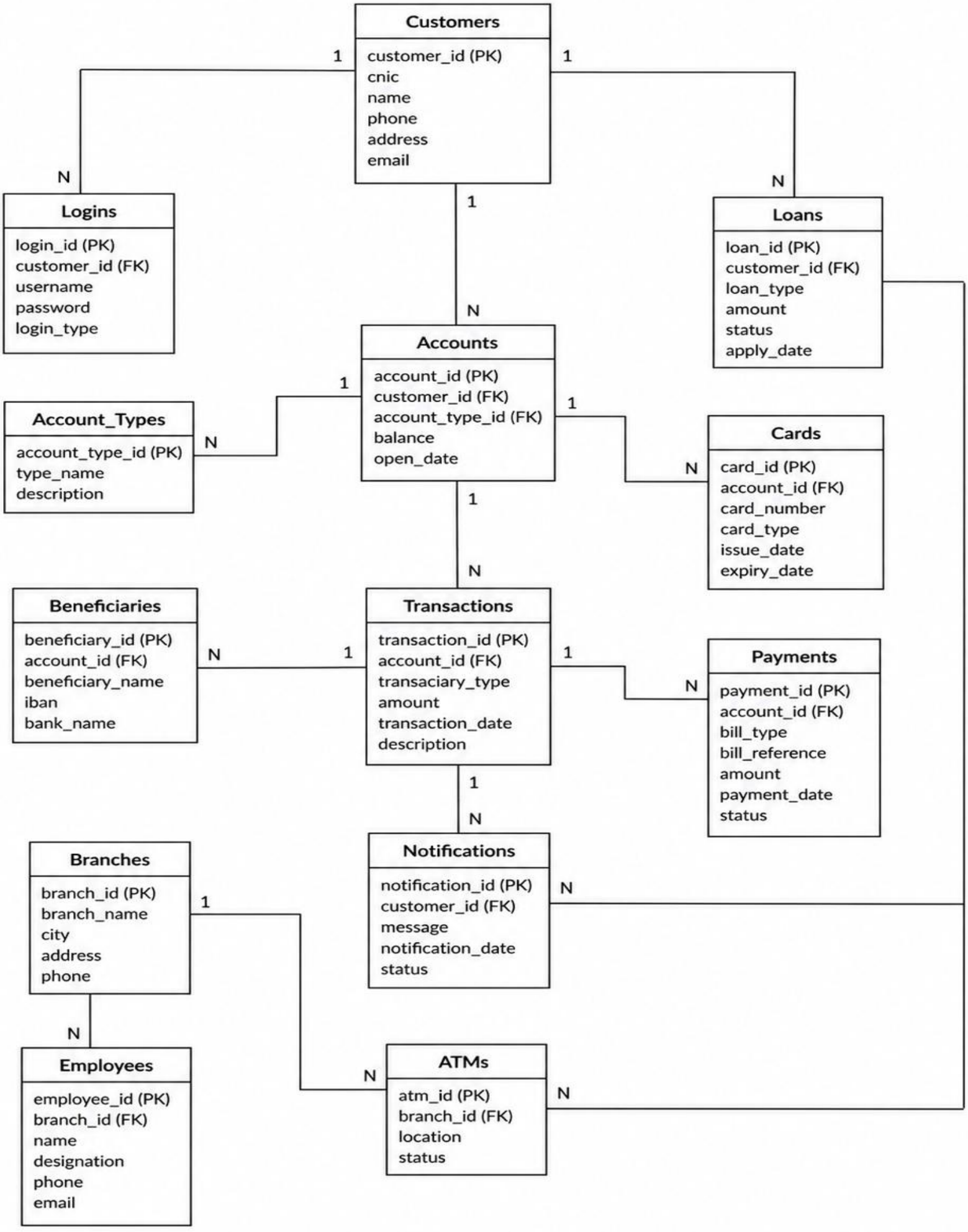
This database-driven application digitizes retail banking in Pakistan by enabling CNIC-registered customers to manage accounts, execute secure IBAN transfers, pay utility bills, and manage loans, ATMs, and debit cards within a centralized system.

Problem Statement

In Pakistan, traditional banking systems still rely heavily on manual processes, paper-based records, and physical branch visits. This results in delays, long queues, and inefficient service delivery.
Customers face difficulties such as:

- Slow transaction processing
- Limited access to banking services in rural areas
- Manual loan processing
- Lack of real-time account management

This system aims to solve these problems by providing a centralized, secure, and automated digital banking platform.


Customers
customer_id (PK)
cnic
name
phone
address
email
Logins
login_id (PK)
customer_id (FK)
username
password
login_type
Loans
loan_id (PK)
customer_id (FK)
loan_type
amount
status
apply_date
Accounts
account_id (PK)
customer_id (FK)
account_type_id (FK)
balance
open_date
Account_Types
account_type_id (PK)
type_name
description
Cards
card_id (PK)
account_id (FK)
card_number
card_type
issue_date
expiry_date
Beneficiaries
beneficiary_id (PK)
account_id (FK)
beneficiary_name
iban
bank_name
Transactions
transaction_id (PK)
account_id (FK)
transaciary_type
amount
transaction_date
description
Payments
payment_id (PK)
account_id (FK)
bill_type
bill_reference
amount
payment_date
status
Notifications
notification_id (PK)
customer_id (FK)
message
notification_date
status
Branches
branch_id (PK)
branch_name
city
address
phone
Employees
employee_id (PK)
branch_id (FK)
name
designation
phone
email
ATMs
atm_id (PK)
branch_id (FK)
location
status
1
N

Entity-Relationship Diagram (ERD) Component Breakdown & Logical Mapping

1. Security Layer (Identity & Access)

- **Logins:** Stores usernames, encrypted passwords, and user access roles.
- **Customers:** Stores permanent profile data (CNIC, name, contact info).
- **Customers ➔ Logins (1:N):** One customer can have multiple secure login methods (e.g., mobile app, web portal, fingerprint ID). Changing a password never impacts their core CNIC profile

2. Core Financial Layer (Accounts & Structure)

- **Account_Types:** Look-up table defining product types (Savings, Current, Islamic PLS).
- **Accounts:** The central ledger tracking account IDs, opening dates, and cash balances.
- **Customers ➔ Accounts (1:N):** One customer can open multiple accounts (e.g., a Current account for shopping and an Islamic account for long-term investments).
- **Account_Types ➔ Accounts (1:N):** Standardizes account behaviors and interest/profit rules.

3. Digital Gateway Layer (Transactions & Ecosystems)

- **Transactions:** The unalterable ledger recording every real-time financial movement.
- **Beneficiaries:** Registry tracking external names and 24-character IBAN codes.
- **Payments:** Utility gateway tracking billers (LESCO, SNGPL) and consumer reference tokens.
- **Accounts ➔ Transactions (1:N):** Every cash movement updates the balance and permanently logs a line item to prevent fraud.
- **Beneficiaries ➔ Transactions (1:N):** Links an outgoing fund transfer directly to a recipient's IBAN for anti-money laundering tracking.

- **Accounts ➔ Payments (1:N):** Isolates utility payments, linking a specific consumer reference bill directly to the paying account.

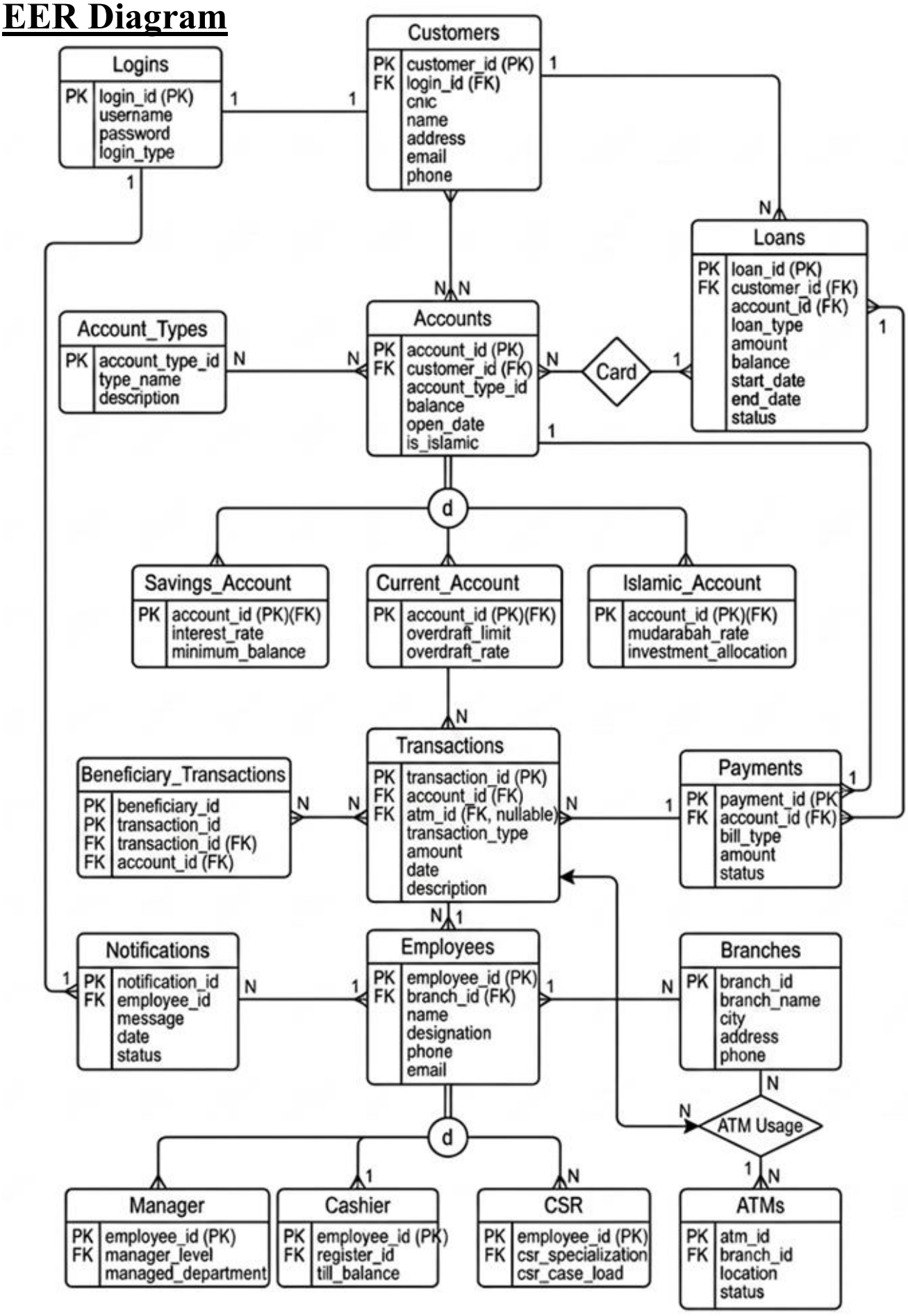
4. Products & Liabilities Layer (Cards & Loans)

- **Cards:** Manages plastic access details (card numbers, debit/credit types, expiries).
- **Loans:** Tracks credit risk exposure (loan types, starting amounts, remaining balances).
- **Accounts ➔ Cards (1:N):** One account can have multiple active cards (e.g., primary debit card + supplementary cards for family members).
- **Customers ➔ Loans (1:N):** Connects debt liabilities to the individual. A single customer can hold a car loan and a business loan simultaneously.

5. Physical Logistics & Alerts

- **Branches:** Tracks physical buildings, cities, and administrative hubs.
- **Employees:** Directory of staff members and their designations.
- **ATMs:** Monitors physical machine locations and cash availability status.
- **Notifications:** Logs real-time security alerts sent to users.
- **Branches ➔ Employees (1:N):** Organizes personnel by binding them to a specific branch location.
- **Branches ➔ ATMs (1:N):** Assigns hardware maintenance and cash refills to local branch teams.
- **ATMs ➔ Transactions (1:N):** Maps cash withdrawals to the exact machine that dispensed the paper currency for forensic auditing.
- **Customers ➔ Notifications (1:N):** Triggers automated system logs (SMS/Email alerts) whenever account activity occurs.

EER Diagram


Customers
PK customer_id (PK)
FK login_id (FK)
cnic
name
address
email
phone
Logins
PK login_id (PK)
username
password
login_type
Loans
PK loan_id (PK)
FK customer_id (FK)
FK account_id (FK)
loan_type
amount
balance
start_date
end_date
status
Account_Types
PK account_type_id
type_name
description
Accounts
PK account_id (PK)
FK customer_id (FK)
account_type_id
balance
open_date
is_islamic
Card
d
Savings_Account
PK account_id (PK)(FK)
interest_rate
minimum_balance
Current_Account
PK account_id (PK)(FK)
overdraft_limit
overdraft_rate
Islamic_Account
PK account_id (PK)(FK)
mudarabah_rate
investment_allocation
Transactions
PK transaction_id (PK)
FK account_id (FK)
FK atm_id (FK, nullable)
transaction_type
amount
date
description
Beneficiary_Transactions
PK beneficiary_id
PK transaction_id
FK transaction_id (FK)
FK account_id (FK)
Payments
PK payment_id (PK)
FK account_id (FK)
bill_type
amount
status
Notifications
PK notification_id
FK employee_id
message
date
status
Employees
PK employee_id (PK)
FK branch_id (FK)
name
designation
phone
email
Branches
PK branch_id
branch_name
city
address
phone
ATM Usage
d
Manager
PK employee_id (PK)
FK manager_level
managed_department
Cashier
PK employee_id (PK)
FK register_id
till_balance
CSR
PK employee_id (PK)
FK csr_specialization
csr_case_load
ATMs
PK atm_id
FK branch_id
location
status

Enhanced Entity-Relationship (EER) Specialization & Relational Mapping

1. Enhanced Security & Identity Subsystem

- **Logins:** Holds strictly authentication markers (username, encrypted password, login_type).
- **Customers:** Holds persistent legal identities (cnic, name, address, email, phone).
- **Logins ➔ Customers (1:1):** *Note the change here!* It is now a strict **1-to-1 relationship**. One unique legal customer maps to exactly one secure login record. This perfectly isolates sensitive authentication data from general customer care views.

2. The Accounts Superclass & Disjoint Inheritance

- **Accounts (Superclass):** The generalized parent table tracking core financial data shared by *all* bank accounts (account_id, balance, open_date, is_islamic).
- **The Disjoint Circle (d):** Specifies that these account categories are completely mutually exclusive. An account ID can belong to one, and *only one*, subclass at any given time.
- **Savings_Account (Subclass):** Inherits its primary key from Accounts and handles conventional growth rules (interest_rate, minimum_balance).
- **Current_Account (Subclass):** Inherits from Accounts and stores checking-specific risk metrics (overdraft_limit, overdraft_rate).
- **Islamic_Account (Subclass):** Inherits from Accounts and routes data under Shariah rules (mudarabah_rate, investment_allocation).
- **Account_Types ➔ Accounts (1:N):** Standardizes validation rules across the superclass ledger before data routes to the specific subclasses.
- **Customers ➔ Accounts (1:N):** A single customer can own multiple distinct account numbers across different subclass styles.

3. Operational Transactions & External Gateways

- **Transactions:** The core unalterable transactional engine logging financial amounts and real-time ledger timestamps.
- **Beneficiary_Transactions:** A specialized bridge table handling inter-bank transfers. It ties a unique transaction directly to an external recipient's name and **IBAN code**.
- **Payments:** A gateway isolating retail utility bill collections (like LESCO or SNGPL tokens).
- **Accounts ➔ Transactions (1:N):** One account record can safely track infinite real-time cash ledger events.
- **Transactions ➔ Beneficiary_Transactions (1:N):** Ensures any outward online fund transfer leaves a permanent, legally auditable trail for anti-money laundering compliance.
- **Accounts ➔ Payments (1:N):** Links bill payment references safely back to the originating master balance.

4. Products, Assets, & Risk Control (Cards & Loans)

- **Cards:** Handles physical card details (Visa/Mastercard tracking, expiry dates, card types).
- **Loans:** Monitors the bank's active risk exposure (types, total amounts, outstanding balances).
- **Accounts ➔ Cards (1:N):** One central bank account can handle multiple active plastic devices (e.g., a primary card and supplementary cards for family).
- **Customers ➔ Loans (1:N):** Connects legal financial debt liability directly to the customer's permanent profile.
- **Loans ➔ Payments (1:1):** Allows automated direct-debit structures where a specific loan repayment can be targeted straight from the client's account routing.

5. Branch Infrastructure & Employee Specialized Hierarchy

- **Branches:** Tracks physical banking infrastructure buildings, local cities, and phone nodes.
- **ATMs:** Tracks independent automated teller machines deployed under a branch.
- **Notifications:** Administrative communication log sending alerts directly to users.
- **Employees (Superclass):** The shared parent table for all bank staff (employee_id, basic details, payroll routing designation).
- **The Disjoint Circle (d):** Enforces a strict operational split. An employee ID can only belong to one specialized role.
- **Manager (Subclass):** Inherits from Employees; tracks executive leadership metrics (manager_level, managed_department).
- **Cashier (Subclass):** Inherits from Employees; strictly locked to frontline cash operations (register_id, active physical till_balance).
- **CSR (Subclass):** Inherits from Employees; tracks consumer support performance indicators (csr_specialization, active csr_case_load).
- **Branches ➔ Employees (1:N):** Binds staff records straight to their physical home office location.
- **Branches ➔ ATMs (1:N):** Pinpoints which physical team handles a hardware machine's maintenance.
- **ATMs ➔ Transactions (1:N via 'ATM Usage'):** Essential for forensic auditing. Traces a physical cash withdrawal back to the exact physical machine that dispensed the paper rupees.
- **Employees ➔ Notifications (1:N):** Logs exactly which staff member or system automated service sent a specific alert to a client.

Relational Schema

Below is the list of relational tables relevant to the Online Banking Management System. Each entity is represented as a table with its attributes, where primary keys uniquely identify records and foreign keys are used to establish relationships between different tables. This structure ensures proper organization of data, reduces redundancy, and maintains consistency across the database.

1. Customers

- customer_id (PK)
- cnic
- name
- phone
- address
- email

2. Logins

- login_id (PK)
- customer_id (FK → Customers.customer_id)
- username
- password
- login_type

3. Accounts

- account_id (PK)
- customer_id (FK → Customers.customer_id)
- account_type_id (FK → Account_Types.account_type_id)
- balance
- open_date

4. Account_Types

- account_type_id (PK)
- type_name
- description

5. Transactions

- transaction_id (PK)
- account_id (FK → Accounts.account_id)
- transaction_type
- amount
- transaction_date
- description

6. Beneficiaries

- beneficiary_id (PK)
- account_id (FK → Accounts.account_id)
- beneficiary_name
- iban
- bank_name

7. Payments

- payment_id (PK)
- account_id (FK → Accounts.account_id)
- bill_type
- bill_reference
- amount
- payment_date
- status

8. Cards

- card_id (PK)
- account_id (FK → Accounts.account_id)
- card_number
- card_type
- issue_date
- expiry_date

9. Loans

- loan_id (PK)

- customer_id (FK → Customers.customer_id)
- loan_type
- amount
- status
- apply_date

10. Notifications

- notification_id (PK)
- customer_id (FK → Customers.customer_id)
- message
- notification_date
- status

11. Branches

- branch_id (PK)
- branch_name
- city
- address
- phone

12. Employees

- employee_id (PK)
- branch_id (FK → Branches.branch_id)
- name
- designation
- phone
- email

13. ATMs

- atm_id (PK)
- branch_id (FK → Branches.branch_id)
- location
- status

Normalization

Introduction

Normalization is the formal process of organizing a database to minimize redundancy and eliminate anomalies such as insertion, deletion, and update problems. In the Online Banking Management System, normalization is applied to ensure that all banking data (customers, accounts, transactions, loans, branches, etc.) is stored in a structured and consistent manner. The database is normalized up to Third Normal Form (3NF).

Unnormalized Form (UNF)

Initially, banking data may exist in a single combined structure where multiple entities are stored together.

UNF Relation:

BANK_SYSTEM(
customer_id,
cnic,
customer_name,
phone,
address,
email,
account_id,
account_type,
balance,
open_date,
transaction_id,
transaction_type,
amount,
transaction_date,
loan_id,
loan_type,
loan_amount,
branch_name,
branch_city,

employee_name
)

Problems in UNF:

- Repetition of customer and account data
- Multiple values mixed in one table
- High redundancy
- Update anomalies (same customer repeated multiple times)

FIRST NORMAL FORM (1NF)

A relation is in 1NF when:

- All attributes are atomic (no repeating groups)
- Each record is unique

1NF Decomposition:

CUSTOMERS
Customers(customer_id, cnic, name, phone, address, email)
ACCOUNTS
Accounts(account_id, customer_id, account_type_id, balance, open_date)
TRANSACTIONS
Transactions(transaction_id, account_id, transaction_type, amount, transaction_date, description)
LOANS
Loans(loan_id, customer_id, loan_type, amount, status, apply_date)
BRANCHES
Branches(branch_id, branch_name, city, address, phone)
EMPLOYEES
Employees(employee_id, branch_id, name, designation, phone, email)
CARDS
Cards(card_id, account_id, card_number, card_type, issue_date, expiry_date)
PAYMENTS
Payments(payment_id, account_id, bill_type, bill_reference, amount, payment_date, status)
BENEFICIARIES
Beneficiaries(beneficiary_id, account_id, beneficiary_name, iban, bank_name)

NOTIFICATIONS
Notifications(notification_id, customer_id, message, notification_date, status)
ATMS
ATMs(atm_id, branch_id, location, status)
ACCOUNT_TYPES
Account_Types(account_type_id, type_name, description)

SECOND NORMAL FORM (2NF)

A relation is in 2NF if:

- It is already in 1NF
- No partial dependency exists (non-key attributes depend on full primary key)

Functional Dependencies in System:

- customer_id → cnic, name, phone, address, email
- account_id → customer_id, account_type_id, balance, open_date
- transaction_id → account_id, transaction_type, amount, transaction_date
- loan_id → customer_id, loan_type, amount, status
- branch_id → branch_name, city, address, phone

THIRD NORMAL FORM (3NF)

A relation is in 3NF if:

- It is in 2NF
- No transitive dependency exists

Transitive Dependency Removal in System:
1. Account Types Separation
Instead of storing:
account_type_name inside Accounts
We created:
Account_Types(account_type_id, type_name, description)

2. Branch Separation
Branch details are not stored inside Employees or ATMs:
Branches(branch_id, branch_name, city, address, phone)

3. Independent Banking Modules
Each entity is independent:
Transactions → separate from Accounts
Loans → separate from Customers
Payments → separate from Accounts
Cards → separate table
Notifications → separate table

Final Conclusion

The Online Banking Management System is successfully normalized up to Third Normal Form (3NF). The normalization process removes redundancy, eliminates insertion, deletion, and update anomalies, and ensures a well-structured relational database design. The final design supports efficient storage and retrieval of banking data while maintaining data integrity across all entities.

Relational Algebra

What is Relational Algebra?

Relational Algebra is a query language that uses mathematical symbols to retrieve data from database tables. It helps us understand how data is selected, filtered, and combined before writing SQL queries.

Basic Operations Used

1. Selection (σ)

Used to filter rows based on a condition.

2. Projection (π)

Used to display only selected columns.

3. Natural Join (⋈)

Used to combine data from two related tables.

The 5 Core Project Queries

1. Searching for a Specific Customer

Mathematical Form:

σ customer_id = 1 (Customers)

Explanation: Displays the customer whose ID is 1.

2. Customer Phone Directory

Mathematical Form:

π name, phone (Customers)

Explanation: Shows only customer names and phone numbers.

3. Finding Low-Balance Accounts

Mathematical Form:

σ balance < 5000 (Accounts)

Explanation: Displays accounts having balance less than 5000.

4. Matching Customers with Accounts

Mathematical Form:

Customers ⋈ Accounts

Explanation: Combines customer and account information.

5. Viewing Transaction History

Mathematical Form:

σ account_id = 1002 (Transactions)

Explanation: Shows all transactions for account 1002.

Core Tables

```sql
-- 1. Branches Table: Bank ki saari physical branches ka data save karne k liye
CREATE TABLE Branches (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    address VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL
);

-- 2. Logins Table: Security aur login data ko baqi records se alag rakhne k liye
CREATE TABLE Logins (
    login_id INT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    login_type ENUM('Customer', 'Employee') NOT NULL
);

-- 3. Customers Table: Customers ki personal details aur unka login account map karne k liye
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    login_id INT UNIQUE,
    cnic VARCHAR(15) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL,
    FOREIGN KEY (login_id) REFERENCES Logins(login_id) ON DELETE CASCADE
);

-- 4. Account Types Lookup Table: Accounts ki mukhtalif types ki settings save karne k liye
CREATE TABLE Account_Types (
    account_type_id INT PRIMARY KEY,
    type_name VARCHAR(50) NOT NULL,
    description VARCHAR(255)
);

-- 5. Accounts Superclass Table: Tamam accounts ka main ledger jahan bunyadi data save hota hai
CREATE TABLE Accounts (
    account_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    account_type_id INT NOT NULL,
    balance DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    open_date DATE NOT NULL,
    is_islamic BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT chk_positive_balance CHECK (balance >= 0.00),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE RESTRICT,
    FOREIGN KEY (account_type_id) REFERENCES Account_Types(account_type_id)
);
```

Phase 2 — Class Table Inheritance: Account Subclasses

```sql
-- 6. Savings Account Subclass: Conventional savings accounts ka sood/interest rate save karne k
liye
CREATE TABLE Savings_Account (
    account_id INT PRIMARY KEY,
    interest_rate DECIMAL(5,2) NOT NULL,
    minimum_balance DECIMAL(10,2) NOT NULL DEFAULT 5000.00,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE CASCADE
);
```

```sql
-- 7. Current Account Subclass: Business aur current accounts ki overdraft limit manage karne k liye
CREATE TABLE Current_Account (
    account_id INT PRIMARY KEY,
    overdraft_limit DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    overdraft_rate DECIMAL(5,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE CASCADE
);


-- 8. Islamic Account Subclass: Shariah compliant/Mudarabah accounts ki details k liye
CREATE TABLE Islamic_Account (
    account_id INT PRIMARY KEY,
    mudarabah_rate DECIMAL(5,2) NOT NULL,
    investment_allocation VARCHAR(150) NOT NULL,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE CASCADE
);
```

Phase 3 — Staff Hierarchy & Inheritance

```sql
-- 9. Base Employees Table: Tamam bank staff ki general details save karne k liye
CREATE TABLE Employees (
    employee_id INT PRIMARY KEY,
    branch_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    designation VARCHAR(50) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id)
);


-- 10. Manager Subclass: Branch managers ki specialization aur unka level track karne k liye
CREATE TABLE Manager (
    employee_id INT PRIMARY KEY,
    manager_level VARCHAR(50) NOT NULL,
    managed_department VARCHAR(50) NOT NULL,
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id) ON DELETE CASCADE
);


-- 11. Cashier Subclass: Frontline counter cashiers ka cash register aur till balance manage karne k liye
CREATE TABLE Cashier (
    employee_id INT PRIMARY KEY,
    register_id VARCHAR(50) UNIQUE NOT NULL,
    till_balance DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id) ON DELETE CASCADE
);


-- 12. CSR Subclass: Customer Services Representatives ka current load aur cases check karne k liye
CREATE TABLE CSR (
    employee_id INT PRIMARY KEY,
    csr_specialization VARCHAR(50) NOT NULL,
    csr_case_load INT NOT NULL DEFAULT 0,
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id) ON DELETE CASCADE
);
```

Phase 4 — Transactions, Channels & Utilities

```sql
-- 13. ATMs Infrastructure Table: Har branch k ATM hardware aur unka status track karne k liye
```

```
CREATE TABLE ATMs (
    atm_id INT PRIMARY KEY,
    branch_id INT NOT NULL,
    location VARCHAR(150) NOT NULL,
    status ENUM('Active', 'Maintenance', 'Out of Cash') NOT NULL DEFAULT 'Active',
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id)
);


-- 14. Master Ledger Transactions Table: Bank main hone wali har choti bari transaction ka unalterable record
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY,
    account_id INT NOT NULL,
    atm_id INT DEFAULT NULL,
    transaction_type ENUM('ATM Cash Withdrawal', 'Cash Deposit', 'Online Transfer',
                          'IBFT Transfer', 'Profit Deposit', 'Cheque Deposit') NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    description VARCHAR(255),
    CONSTRAINT chk_positive_tx CHECK (amount > 0.00),
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE RESTRICT,
    FOREIGN KEY (atm_id) REFERENCES ATMs(atm_id)
);


-- 15. Beneficiary Third-Party Bridge Table: Dusray banks main transfer (IBFT) k liye aur recipient tracking k liye
CREATE TABLE Beneficiary_Transactions (
    beneficiary_id INT PRIMARY KEY AUTO_INCREMENT,
    transaction_id INT NOT NULL,
    account_id INT NOT NULL,
    beneficiary_name VARCHAR(100) NOT NULL,
    iban VARCHAR(34) NOT NULL,
    FOREIGN KEY (transaction_id) REFERENCES Transactions(transaction_id) ON DELETE CASCADE,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id)
);


-- 16. Ecosystem Consumer Payments: Utility bills (LESCO, SNGPL waghaira) ki integration k liye
CREATE TABLE Payments (
    payment_id INT PRIMARY KEY,
    account_id INT NOT NULL,
    bill_type VARCHAR(50) NOT NULL,
    bill_reference VARCHAR(50) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status ENUM('Pending', 'Paid', 'Failed') NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE RESTRICT
);
```

Phase 5 — Cards, Liabilities & Notifications

```
-- 17. Cards Management Table: Debit aur Credit cards ka data aur unki validity save karne k liye
CREATE TABLE Cards (
    card_id INT PRIMARY KEY,
    account_id INT NOT NULL,
    card_number VARCHAR(16) UNIQUE NOT NULL,
    card_type VARCHAR(50) NOT NULL,
    issue_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id) ON DELETE CASCADE
);


-- 18. Commercial Loans Table: Customers k liye loans aur bache hue outstanding balance track karne k liye
CREATE TABLE Loans (
```

```
    loan_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    account_id INT NOT NULL,
    loan_type VARCHAR(100) NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    balance DECIMAL(15,2) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status ENUM('Pending', 'Approved', 'Rejected', 'Closed') NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id)
);


-- 19. Administrative Logs / System Notifications: Alerts aur safety messages ka record rakhne k
liye
CREATE TABLE Notifications (
    notification_id INT PRIMARY KEY,
    employee_id INT NOT NULL,
    message TEXT NOT NULL,
    date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status ENUM('Sent', 'Failed') NOT NULL DEFAULT 'Sent',
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id) ON DELETE CASCADE
);
```

4. Data Seeding

```
-- Branches ka sample data entry
INSERT INTO Branches (branch_id, branch_name, city, address, phone) VALUES
(1, 'DHA Phase 6 Branch', 'Lahore', 'Main Boulevard, DHA Phase 6', '042-35691111'),
(2, 'I.I. Chundrigar Head Branch', 'Karachi', 'Main I.I. Chundrigar Road', '021-32211111'),
(3, 'F-7 Markaz Branch', 'Islamabad', 'Jinnah Avenue, F-7 Markaz', '051-2277111');


-- Account Categories ka entry data
INSERT INTO Account_Types (account_type_id, type_name, description) VALUES
(100, 'Savings', 'Standard profit bearing account with variable interest thresholds'),
(200, 'Current', 'Standard business checking account with overdraft facilities'),
(300, 'Islamic PLS', 'Shariah-compliant profit and loss sharing account under Mudarabah rules');


-- Logins records (10 Customers aur 3 Employees k liye security access data)
INSERT INTO Logins (login_id, username, password, login_type) VALUES
(10, 'kamran_bhatti', 'K@mran786', 'Customer'),
(20, 'ayesha_malik', 'Ayesha#2026', 'Customer'),
(30, 'zayan_ahmed', 'Zayan$99', 'Customer'),
(40, 'fiona_shah', 'FiOnA!@#', 'Customer'),
(50, 'bilal_dar', 'BilalDar88', 'Customer'),
(60, 'zainab_baloch', 'ZainabKhan1', 'Customer'),
(70, 'mustafa_junejo', 'MustafaJu90', 'Customer'),
(80, 'hina_rabbani', 'HinaR@b12', 'Customer'),
(90, 'umar_farooq', 'UmarFarooq5', 'Customer'),
(100, 'sadia_baig', 'SadiaBaig00', 'Customer'),
(901, 'tariq_mgr', 'MgrPass123', 'Employee'),
(902, 'sajid_csh', 'CshPass123', 'Employee'),
(903, 'amna_csr', 'CsrPass123', 'Employee');


-- 10 Pakistani Customers ka legal data entries
INSERT INTO Customers (customer_id, login_id, cnic, name, address, email, phone) VALUES
(1, 10, '35202-1982731-1', 'Kamran Bhatti', 'Gulberg III, Lahore', 'kamran.bhatti@gmail.com',
'03001234567'),
(2, 20, '42201-8873625-2', 'Ayesha Malik', 'Clifton Block 5, Karachi', 'ayesha.malik@yahoo.com',
'03219876543'),
(3, 30, '61101-0928374-3', 'Zayan Ahmed', 'G-11, Islamabad', 'zayan.ahmed@outlook.com',
'03335551122'),
```

```
(4, 40, '35201-7762514-4', 'Fiona Shah', 'Model Town, Lahore', 'fiona.shah@gmail.com',
'03014445556'),
(5, 50, '82301-4435261-5', 'Bilal Dar', 'Muzaffarabad Sector C', 'bilal.dar@hotmail.com',
'03459998887'),
(6, 60, '54401-9988776-6', 'Zainab Baloch', 'Samungli Road, Quetta', 'zainab.baloch@gmail.com',
'03138887776'),
(7, 70, '41303-1234567-7', 'Mustafa Junejo', 'Saddar, Hyderabad', 'mustafa.junejo@yahoo.com',
'03221112223'),
(8, 80, '37405-7654321-8', 'Hina Rabbani', 'Saddar, Rawalpindi', 'hina.rabbani@outlook.com',
'03347776665'),
(9, 90, '31202-1122334-9', 'Umar Farooq', 'Model Town, Bahawalpur', 'umar.farooq@gmail.com',
'03023334445'),
(10, 100, '13302-5566778-1', 'Sadia Baig', 'Main Bazar, Abbottabad', 'sadia.baig@gmail.com',
'03156667778');


-- 12 Master Accounts ka primary database record entries
INSERT INTO Accounts (account_id, customer_id, account_type_id, balance, open_date, is_islamic)
VALUES
(501, 1, 100, 450000.00, '2024-03-15', FALSE),
(502, 2, 200, 75000.00, '2025-01-10', FALSE),
(503, 3, 300, 1250000.00, '2025-05-20', TRUE),
(504, 4, 200, 19500.00, '2025-06-01', FALSE),
(505, 5, 100, 620000.00, '2025-07-12', FALSE),
(506, 6, 300, 89000.00, '2025-08-19', TRUE),
(507, 7, 200, 45000.00, '2025-09-05', FALSE),
(508, 8, 100, 3100000.00, '2025-10-01', FALSE),
(509, 9, 300, 15000.00, '2025-11-22', TRUE),
(510, 10, 200, 550000.00, '2025-12-25', FALSE),
(511, 1, 200, 12000.00, '2026-01-05', FALSE),
(512, 2, 100, 950000.00, '2026-02-14', FALSE);


-- Savings account subclass details ki entries
INSERT INTO Savings_Account (account_id, interest_rate, minimum_balance) VALUES
(501, 11.50, 5000.00), (505, 12.00, 5000.00), (508, 13.25, 10000.00), (512, 11.00, 5000.00);


-- Current account details aur overdraft configuration ka data
INSERT INTO Current_Account (account_id, overdraft_limit, overdraft_rate) VALUES
(502, 100000.00, 14.50), (504, 20000.00, 15.00), (507, 50000.00, 14.00),
(510, 250000.00, 14.00), (511, 10000.00, 15.00);


-- Islamic Shariah pools aur investments mapping records
INSERT INTO Islamic_Account (account_id, mudarabah_rate, investment_allocation) VALUES
(503, 9.20, 'Government Sukuk Energy Bonds Pool'),
(506, 8.80, 'Agriculture Development Shariah Fund'),
(509, 9.00, 'Corporate Murabaha Trade Pool');


-- Staff key management ki entries
INSERT INTO Employees (employee_id, branch_id, name, designation, phone, email) VALUES
(701, 1, 'Tariq Javed', 'Branch Manager', '03008441122', 'tariq.javed@bank.pk'),
(702, 1, 'Sajid Raza', 'Main Teller Cashier', '03454449988', 'sajid.raza@bank.pk'),
(703, 2, 'Amna Farooq', 'Customer Care Advisor', '03126663322', 'amna.farooq@bank.pk');


-- Subclasses records employees hierarchy k hisab se
INSERT INTO Manager (employee_id, manager_level, managed_department) VALUES
    (701, 'VP - Senior Executive', 'Branch Administration');
INSERT INTO Cashier (employee_id, register_id, till_balance) VALUES
    (702, 'LHR-DHA6-CASH01', 345000.00);
INSERT INTO CSR (employee_id, csr_specialization, csr_case_load) VALUES
    (703, 'Retail Consumer Support', 14);


-- Local ATMs locations aur setup entries
INSERT INTO ATMs (atm_id, branch_id, location, status) VALUES
(901, 1, 'DHA Phase 6 Outward Lobby ATM', 'Active'),
(902, 2, 'Chundrigar Ground Floor ATM 1', 'Active');
```

```sql
-- Multi-channel financial transactions logs entry
INSERT INTO Transactions (transaction_id, account_id, atm_id, transaction_type, amount, date, description) VALUES
(801, 501, NULL, 'Online Transfer', 35000.00, '2026-03-01 10:00:00', 'Funds sent to sister mobile wallet'),
(802, 502, 901, 'ATM Cash Withdrawal', 20000.00, '2026-03-02 14:30:00', 'ATM cash withdrawal counter transaction'),
(803, 503, NULL, 'Profit Deposit', 8500.00, '2026-03-05 00:00:00', 'Monthly Shariah Profit payout credit'),
(804, 504, 901, 'ATM Cash Withdrawal', 5000.00, '2026-03-06 18:22:00', 'Late night emergency cash out check'),
(805, 505, NULL, 'IBFT Transfer', 15000.00, '2026-03-07 11:15:00', 'Inter-bank fund transfer payment'),
(806, 506, 902, 'ATM Cash Withdrawal', 10000.00, '2026-03-08 09:40:00', 'Karachi branch ATM cash out'),
(807, 507, NULL, 'Online Transfer', 25000.00, '2026-03-09 13:12:00', 'Payment to vendor account'),
(808, 508, NULL, 'Cheque Deposit', 120000.00, '2026-03-10 12:00:00', 'Clearing transaction credit entry'),
(809, 510, NULL, 'Online Transfer', 45000.00, '2026-03-11 16:45:00', 'Rent transaction transfer'),
(810, 512, NULL, 'Cash Deposit', 50000.00, '2026-03-12 11:00:00', 'Over the counter cash deposit asset');


-- IBFT transfers k recipient names aur IBAN tracking ka data
INSERT INTO Beneficiary_Transactions (beneficiary_id, transaction_id, account_id, beneficiary_name, iban) VALUES
(1, 801, 501, 'Nida Bhatti', 'PK24MNET0000352019827311'),
(2, 805, 505, 'Zeeshan Ali', 'PK09HABB0002991122334455'),
(3, 807, 507, 'Aslam Traders', 'PK77BAHL0001009988776655');


-- Utility consumer tracking entries (LESCO, SNGPL waghaira)
INSERT INTO Payments (payment_id, account_id, bill_type, bill_reference, amount, status) VALUES
(1, 502, 'Electricity (LESCO)', '14112233445566U', 18500.00, 'Paid'),
(2, 503, 'Gas (SNGPL)', '99887766554433G', 4200.00, 'Paid'),
(3, 504, 'Electricity (K-ELECTRIC)', '0400998811223X', 12300.00, 'Paid'),
(4, 507, 'Water (KWSB)', '77665544332211W', 1450.00, 'Paid'),
(5, 510, 'Electricity (IESCO)', '19228833774455E', 22400.00, 'Paid');


-- Plastic debit/credit cards allocation data
INSERT INTO Cards (card_id, account_id, card_number, card_type, issue_date, expiry_date) VALUES
(1, 501, '4214888899990000', 'Visa Gold Debit', '2024-03-16', '2029-03-16'),
(2, 502, '5212333344445555', 'Mastercard Classic', '2025-01-11', '2030-01-11'),
(3, 503, '4214222233334444', 'Visa Platinum', '2025-05-21', '2030-05-21'),
(4, 505, '5212666677778888', 'Mastercard Gold', '2025-07-13', '2030-07-13'),
(5, 508, '4214555566667777', 'Visa Signature', '2025-10-02', '2030-10-02'),
(6, 510, '4214111122223333', 'Visa Platinum Debit', '2025-12-26', '2030-12-26');


-- Credit facilities aur active loans monitoring setup records
INSERT INTO Loans (loan_id, customer_id, account_id, loan_type, amount, balance, start_date, end_date, status) VALUES
(1, 1, 501, 'PM Youth Scheme Car Loan', 2500000.00, 2200000.00, '2024-06-01', '2029-06-01', 'Approved'),
(2, 5, 505, 'Personal Construction Loan', 5000000.00, 5000000.00, '2025-08-01', '2035-08-01', 'Approved'),
(3, 8, 508, 'SME Business Loan', 8000000.00, 7500000.00, '2025-10-15', '2030-10-15', 'Approved');


-- Automated database alerts logs entry
INSERT INTO Notifications (notification_id, employee_id, message, date, status) VALUES
(1, 703, 'Dear Customer, your internet banking profile has been successfully modified.',
 '2026-01-01 09:00:00', 'Sent');


-- Saari tables ka status dekhne k liye confirmation command
SHOW TABLES;
```

Query 1: Live Cash Withdrawal Transaction

Objective

This query demonstrates a real-time ATM cash withdrawal transaction. It records the transaction in the transaction ledger and updates the customer account balance accordingly.

SQL Query

-- Kamran Bhatti withdrawing 5,000 PKR from ATM

```
349
350     -- Kamran Bhatti (Account 501) withdrawing 5,000 PKR from an ATM
351     INSERT INTO Transactions (transaction_id, account_id, atm_id, transaction_type, amount, date, description)
352     VALUES (811, 501, 901, 'ATM Cash Withdrawal', 5000.00, NOW(), 'Class Demo Live Withdrawal');
353
354     -- STEP 1: Deduct the 5,000 PKR from Kamran's master account balance
355     UPDATE Accounts
356     SET balance = balance - 5000.00
357     WHERE account_id = 501;
358
359     -- STEP 2: Show the teacher the newly updated balance (It will drop to 445,000.00)
360     SELECT account_id, customer_id, balance
361     FROM Accounts
362     WHERE account_id = 501;
363
364     -- STEP 3: Show the teacher that the transaction row was perfectly logged at the bottom
365     SELECT transaction_id, account_id, transaction_type, amount, description
366     FROM Transactions
367     WHERE transaction_id = 811;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

account_id	customer_id	balance
501	1	445000.00
NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

transaction_id	account_id	transaction_type	amount	description
811	501	ATM Cash Withdrawal	5000.00	Class Demo Live Withdrawal
NULL	NULL	NULL	NULL	NULL

Explanation

This query demonstrates:

- Real-time transaction processing
- Ledger management
- Account balance updating

- ATM integration functionality

Query 2: Customer Debit Card Verification

Objective

This query retrieves customer card information and verifies active debit card details associated with a bank account.

SQL Query

```
369 •   SELECT c.name, a.account_id, cd.card_number, cd.card_type, cd.expiry_date
370     FROM Customers c
371     JOIN Accounts a ON c.customer_id = a.customer_id
372     JOIN Cards cd ON a.account_id = cd.account_id
373     WHERE c.name = 'Kamran Bhatti';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	account_id	card_number	card_type	expiry_date
Kamran Bhatti	501	4214888899990000	Visa Gold Debit	2029-03-16

Explanation

This query demonstrates:

- Multi-table joins
- Card management integration
- Customer-account-card relationship handling

Query 3: Islamic Banking Accounts Report

Objective

This query displays all Islamic banking accounts along with their Mudarabah profit-sharing details and investment allocations.

SQL Query

```
375 •   SELECT a.account_id, c.name, a.balance, ia.mudarabah_rate, ia.investment_allocation
376     FROM Accounts a
377     JOIN Customers c ON a.customer_id = c.customer_id
378     JOIN Islamic_Account ia ON a.account_id = ia.account_id
379     WHERE a.is_islamic = TRUE;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

account_id	name	balance	mudarabah_rate	investment_allocation
503	Zayan Ahmed	1250000.00	9.20	Government Sukuk Energy Bonds Pool
506	Zainab Baloch	89000.00	8.80	Agriculture Development Shariah Fund
509	Umar Farooq	15000.00	9.00	Corporate Murabaha Trade Pool

Explanation

This query demonstrates:

- Islamic banking implementation
- Class table inheritance
- Specialized Islamic account management

Query 4: Utility Bills Processing Summary

Objective

This query calculates the total number of utility bills processed and total payment amounts collected by the banking system.**SQL Query**

```
392 •   SELECT bill_type, COUNT(*) AS 'Total Bills Processed', SUM(amount) AS 'Total Revenue Collected'
393     FROM Payments
394     WHERE status = 'Paid'
395     GROUP BY bill_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_type	Total Bills Processed	Total Revenue Collected
Electricity (LESCO)	1	18500.00
Gas (SNGPL)	1	4200.00
Electricity (K-ELECTRIC)	1	12300.00
Water (KWSB)	1	1450.00
Electricity (IESCO)	1	22400.00

Explanation

This query demonstrates:
Aggregation functions

- Digital payment ecosystem
- Utility bill integration
- Financial reporting analytics